

P2500R0

C++17 parallel algorithms and P2300

Published Proposal, 2022-10-15

Author:

[Ruslan Arutyunyan](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper provides the facilities to integrate [\[P2300R5\]](#) with C++17 parallel algorithms

Table of Contents

1 Motivation

2 Proposed API

2.1 Parallel algorithms CPO

2.1.1 Why `scheduler`?

2.1.2 Alternative API

2.2 `execute_on`

2.3 `policy_aware_scheduler`

2.4 `execution_policy` concept

2.5 Open question

3 Further exploration

References

Informative References

§ 1. Motivation

C++17 parallel algorithms, together with executions policies, were a good start for parallel computation in C++ standard. However, they don't have explicit way to specify what hardware the algorithm should use to be executed on.

In C++ standard we have execution policies that represent "how" the particular algorithm should be executed, in other words, they provide the semantical guarantees for the user callable objects passed to parallel algorithms. Without having other facilities in the C++ standard library execution policies tend to be used to combine both semantics of "how" and "where" the code should be executed.

[P2300R5] introduces `scheduler` concept that represents the execution context. It's more flexible abstraction comparing with using the execution policies for answering "where" the code should be executed because `scheduler` is tightly connected to the hardware it sends work to.

Since [P2300R5] is targeted to C++26 we also should answer the question how the rest of C++ standard library would interoperate with the schedulers/senders/receiver mechanism.

P2500R0 is targeted to C++26 and is intended to answer the question how C++17 parallel algorithms support [P2300R5] facilities.

§ 2. Proposed API

§ 2.1. Parallel algorithms CPO

`std::for_each` is used as a reference. When the design has a consensus it can be applied to all parallel algorithms.

The proposed API for C++17 parallel algorithms is a customization point object with the following signature of `operator()`:

```
struct __for_each
{
    template <std::policy_aware_scheduler Scheduler, typename It, typename
    void operator()(Scheduler s, It b, It e, Callable c) const
    {
        if constexpr (std::tag_invocable<__for_each, Scheduler, It, It, Cal
        {
            std::tag_invoke(*this, s, b, e, c);
        }
    }
}
```

```

        else
        {
            // default implementation
        }
    }
};

inline constexpr __for_each for_each;

```

See § 2.3 [policy_aware_scheduler](#) section for more details about `policy_aware_scheduler` concept.

The implementation should invoke the customization, if exists. Otherwise, the default generic implementation is called. That allows customizing every particular algorithm by `scheduler` vendor, if necessary.

Note: P2500 is supposed to use the same customization point mechanism as [P2300R5] does (currently `std::tag_invoke`).

Eventually, the API above should be combined with § 2.2 [execute_on](#) and the call would look like:

```
for_each(execute_on(scheduler, std::execution::par), begin, end, callable);
```

§ 2.1.1. Why `scheduler`?

The algorithms should accept the `scheduler` to be able to get as many senders as they need to be able to build a dependency graph they want.

§ 2.1.2. Alternative API

Alternative API might look like having both `scheduler` and `execution_policy` as `operator()` parameters.

```

struct __for_each
{
    template <std::policy_aware_scheduler Scheduler, std::execution_policy

```

```

        typename It, typename Callable>
        void operator()(Scheduler s, ExecutionPolicy policy, It b, It e, Callable f) {
    };

    inline constexpr __for_each for_each;

```

However (IMHO), it complicates the API and still requires `scheduler` to check if it can work with passed execution policy object but on later stage (after resolving the algorithm call) and requires either something like `execute_on` (see § 2.2 `execute_on`) underneath or direct API from `scheduler` (or execution policy) for that kind of checking.

§ 2.2. `execute_on`

`execute_on` is the customization point that serves the purpose to tie `scheduler` and `execution_policy`.

It's up to `scheduler` customization to check if it can work with the passed execution policy.

```

struct __execute_on {
    policy_aware_scheduler auto operator()(scheduler auto sched,
                                           execution_policy auto policy) const {
        {
            return std::tag_invoke(*this, sched, policy);
        }
    }
};

inline constexpr __execute_on execute_on;

```

`execute_on` might have the default implementation but it's a open question what the behavior it should implement. See § 2.5 [Open question](#) for more details.

§ 2.3. `policy_aware_scheduler`

`policy_aware_scheduler` is a concept for parallel algorithms that represents a combined `scheduler` and `execution_policy` entity. It allows to get both execution policy type and execution policy object parallel algorithm is called with.

```
template <typename S>
concept policy_aware_scheduler = scheduler<S> && requires (S s) {
    typename S::base_scheduler_type;
    typename S::policy_type;
    { s.get_policy() } -> execution_policy; // requires to allow specialization
                                           // of execution_policy on the type
                                           // Also might require to make p
};
```

See § 2.4 `execution_policy` concept for more details about `execution_policy` concept.

Customizations of the parallel algorithms can reuse the existing implementation of parallel algorithms with `ExecutionPolicy` template parameter for "known" `base_scheduler_type` type.

§ 2.4. `execution_policy` concept

The execution policy is optional if we want to constraint the return type of (some kind of) `s.get_policy()` method for `policy_aware_scheduler`.

```
template <typename ExecutionPolicy>
concept execution_policy = std::is_execution_policy_v<std::remove_cvref_t<E
```

The potential problem with that concept, thought, is a support of user-defined policies. We might need to allow `is_execution_policy` specialization.

§ 2.5. Open question

- Should `execute_on` have default implementation?
 - If yes, should it advice sequential execution using passed `scheduler` execution resources or the calling thread?

- If no, what a default behavior should it advice for `scheduler` to implement?
- What if the `scheduler` is used in entry point to the binary as a polymorphic (or type-erased) `scheduler`? How would it know that customization appears?
- If `execution_policy` concept is necessary should specialization of `is_execution_policy` be allowed?

§ 3. Further exploration

The author is planning to explore how to specify the set of basic functions (named "parallel backend") the rest of parallel algorithms can be expressed with. It might be a separate paper based on the analysis.

§ References

§ Informative References

[P2300R5]

Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach. ``std::execution``. 22 April 2022.
URL: <https://wg21.link/p2300r5>